

logo_austral.png

MAESTRÍA EN EXPLOTACIÓN DE DATOS Y GESTIÓN DEL CONOCIMIENTO

Universidad Austral

TRABAJO FINAL DE MAESTRÍA

**Evaluación del impacto de la evolución de
arquitecturas de NLP en la clasificación de
texto a escala industrial:**

Un análisis comparativo sobre el Meli Challenge 2019

Autora: Maribel Fraire

Director: Mariano Crosetti

Codirectora: Fernanda Mendez

Mayo de 2026

Resumen

La clasificación automática de texto es una tarea central del Procesamiento de Lenguaje Natural (NLP) que se enmarca dentro del aprendizaje supervisado. En este paradigma, el algoritmo aprende a partir de un conjunto de instancias etiquetadas, ajustando iterativamente los parámetros del modelo mediante la minimización de una función de pérdida, con el objetivo de generalizar y predecir la categoría correcta para nuevas instancias no vistas durante el entrenamiento. Automatizar este tipo de tarea representa un desafío significativo en la industria, dado que transformar datos no estructurados —como los títulos de productos de un e-commerce— en categorías estructuradas es determinante para el funcionamiento de los sistemas de búsqueda y la generación de métricas que permitan la toma de decisiones.

En el contexto de la competencia de Kaggle denominada Meli Challenge 2019, la tarea consiste en clasificar títulos y descripciones de productos que los usuarios del e-commerce han ingresado manualmente en categorías de productos específicas, donde cada título de producto representa una instancia y cada categoría representa una clase posible. Este caso presenta, además, el desafío de que los títulos pueden estar en español o portugués pero el *output* —la categoría— es en inglés.

En este trabajo se propone evaluar el rendimiento de distintos enfoques de clasificación de texto aplicados al dataset del Meli Challenge 2019: clasificación directa mediante *few-shot prompting* con modelos GPT (OpenAI y Groq), embeddings contextuales utilizando BETO (versión en español de BERT, con fine-tuning sobre el dataset completo), embeddings contextuales de OpenAI (text-embedding-3-small) con clasificadores KNN y Random Forest, embeddings estáticos FastText combinados con una red neuronal densa, y la replicación de la solución ganadora de la competencia basada en LSTM/GRU con embeddings FastText pre-entrenados. La comparación se realizará mediante métricas estándar (accuracy, F1-score) y permitirá identificar los factores que explican las diferencias de rendimiento entre enfoques de distinta generación y escala.

La metodología propuesta incluye el preprocesamiento del dataset (limpieza, normalización), la implementación de pipelines completos de clasificación, el análisis comparativo de resultados y la discusión de las limitaciones de reproducibilidad de soluciones basadas en dependencias tecnológicas obsoletas. Los resultados contribuyen a comprender el alcance práctico de cada paradigma (embeddings estáticos, modelos Transformer, LLM) en un problema real de NLP en español.

Palabras clave: Clasificación de texto, Deep Learning, embeddings, Large Language Models (LLM), Machine Learning, Procesamiento de Lenguaje Natural (NLP), Transformers.

Índice

1. Introducción	1
1.1. Contexto del Problema	1
1.2. Objetivo General	1
1.3. Objetivos Específicos	1
2. Marco Teórico	2
2.1. Conceptos Básicos de Machine Learning para Clasificación	2
2.1.1. Aprendizaje Automático	2
2.1.2. Tarea de Clasificación	2
2.1.3. Métricas de Evaluación	3
2.1.4. Generalización	3

2.2.	Procesamiento de Lenguaje Natural (NLP)	4
2.2.1.	Desafíos del Lenguaje Natural	5
2.2.2.	Evolución de Paradigmas en la Clasificación de Texto	5
2.3.	Deep Learning y Redes Neuronales Profundas	6
2.3.1.	Redes Neuronales Profundas (Deep Feedforward Networks)	6
2.3.2.	Descenso por Gradiente Estocástico y Función de Pérdida	7
2.3.3.	LSTM (Long Short-Term Memory)	8
2.3.4.	GRU (Gated Recurrent Unit)	9
2.4.	Representación del Texto: Embeddings	9
2.4.1.	De la Representación Esparsa a los Embeddings Densos	9
2.4.2.	Word2Vec y FastText	9
2.4.3.	Embeddings Contextuales	10
2.5.	Arquitectura Transformer	10
2.5.1.	Tokenización	11
2.5.2.	Embedding Lookup (Búsqueda de Embeddings)	12
2.5.3.	Codificación Posicional (<i>Positional Encoding</i>)	12
2.5.4.	Mecanismo de Autoatención (<i>Self-Attention</i>)	12
2.5.5.	Agregación y Pooling	13
2.5.6.	Red Neuronal Feed-Forward	13
2.6.	Modelos Pre-entrenados para NLP	13
2.6.1.	BERT	13
2.6.2.	BETO: BERT para el Español	14
2.6.3.	GPT y Embeddings de OpenAI	14
2.7.	Grandes Modelos de Lenguaje (LLM)	14
2.7.1.	Definición y Escala	14
2.7.2.	Pre-entrenamiento y Fine-tuning	14
2.7.3.	Encoder vs. Decoder: Relevancia para Clasificación	15
2.8.	Métricas de Evaluación	15
2.9.	Trabajos Relacionados	15
2.9.1.	Clasificación de texto en e-commerce	15
2.9.2.	Categorización automática de productos en e-commerce	16
2.9.3.	Clasificación multiclase de texto a gran escala	16
2.9.4.	Benchmarks comparativos de modelos de NLP	16
3.	Metodología e Implementación	16
3.1.	Entorno de Experimentación	16
3.2.	Dataset	17
3.3.	Preprocesamiento	17
3.4.	Descripción de los Modelos Evaluados	18
3.4.1.	Criterios de Selección y Marco Común	18
3.4.2.	Resumen Comparativo de Enfoques	19
3.5.	Modelo 1 — Clasificación por Prompting con GPT (OpenAI)	19
3.5.1.	Descripción del Enfoque	19

3.5.2.	Preparación del Dataset	20
3.5.3.	Plataformas de Inferencia: OpenAI API y Groq	20
3.5.4.	Modelos Evaluados	20
3.5.5.	Resultados	20
3.5.6.	Análisis de Viabilidad Económica	21
3.6.	Modelo 2 — BETO	21
3.6.1.	Configuración del Entrenamiento	21
3.6.2.	Dependencia de Accelerate	22
3.6.3.	Preparación del Dataset	22
3.6.4.	Requisitos de Software	22
3.6.5.	Validación de Extrapolabilidad de Resultados	22
3.7.	Modelo 3 — OpenAI Embeddings (text-embedding-3-small)	23
3.7.1.	Metodología del Embedding	23
3.7.2.	Descripción de la Solución	23
3.7.3.	Ensayo 1 (03_tp_final_embedding.ipynb)	23
3.7.4.	Ensayo 2 (03_tp_final_embedding_2.ipynb)	24
3.7.5.	Batch API	24
3.8.	Modelo 4 — FastText con Red Neuronal	24
3.9.	Solución Original del Meli Challenge 2019	24
3.9.1.	Limitaciones de Reproducibilidad	24
3.9.2.	Workarounds Implementados	25
3.10.	Limitaciones del Estudio	25
3.11.	Resultados	26
3.11.1.	Métricas de Evaluación	26
3.11.2.	Análisis Comparativo	27
4.	Discusión	27
5.	Conclusiones y Trabajo Futuro	27
5.1.	Conclusiones	27
5.2.	Trabajo Futuro	27
6.	Referencias	27
A.	Código y Repositorio	29

1 Introducción

1.1 Contexto del Problema

En el núcleo del Procesamiento de Lenguaje Natural (NLP) se encuentra un problema fundamental: cómo representar el significado del lenguaje de forma que una computadora pueda razonarlo. Jurafsky y Martin (2024) ilustran esta complejidad señalando que una misma palabra puede tener múltiples sentidos según el contexto, y que un modelo útil del significado debe permitir realizar inferencias para resolver tareas como la clasificación [3]. Esta ambigüedad es el problema operacional central en dominios como el comercio electrónico: el título “funda negra 13 pulgadas” puede corresponder a fundas para celulares, laptops o accesorios de moda, y el modelo debe capturar el significado contextual para resolverlo correctamente. Manning y Schütze (1999) señalan que abordar este problema requiere integrar conocimiento lingüístico con modelos estadísticos de lenguaje [24].

La motivación central de este trabajo es adquirir dominio sobre los conceptos y herramientas que definen el estado del arte en NLP. En particular, tres paradigmas concentran el interés académico y la demanda industrial actual. Las **redes neuronales recurrentes** (RNN, LSTM, GRU) fueron el primer enfoque capaz de capturar dependencias secuenciales en el texto, demostrando que la arquitectura del modelo importa tanto como los datos [7, 8]. La arquitectura **Transformer**, propuesta por Vaswani et al. (2017) [4], reemplazó a las RNN como referencia al introducir el mecanismo de autoatención, que permite procesar secuencias completas en paralelo y capturar relaciones de largo alcance sin degradación del gradiente. Sobre esta base, los **grandes modelos de lenguaje** (LLM) como BERT y GPT, pre-entrenados sobre corpus masivos y adaptables a tareas específicas mediante fine-tuning, constituyen hoy el paradigma dominante en NLP [5, 12]. Aprender a trabajar con estos tres paradigmas en un problema real es el objetivo formativo que orienta este trabajo.

El Meli Challenge 2019 constituye el marco experimental: una competencia real de Kaggle organizada por Mercado Libre que plantea la clasificación de títulos de productos en más de 1.500 categorías [13]. Este problema condensa los desafíos centrales del NLP aplicado —variabilidad léxica, multilingüismo, escala de datos, desbalance de clases [23]— y permite implementar y comparar los tres paradigmas mencionados sobre el mismo dataset, con las mismas métricas y en condiciones reproducibles. La solución ganadora de la competencia alcanzó un score de 0.91733 en el Public Leaderboard, combinando embeddings FastText con capas LSTM y GRU.

1.2 Objetivo General

Evaluar el rendimiento de enfoques de clasificación de texto —embeddings estáticos, aprendizaje profundo y modelos basados en Transformers— en el marco del Meli Challenge 2019, identificando los factores técnicos y metodológicos que explican las diferencias de rendimiento entre modelos de distinta generación y escala.

1.3 Objetivos Específicos

1. Analizar las características y el alcance de los modelos de clasificación de texto utilizados en la práctica: embeddings estáticos, aprendizaje profundo y modelos basados en la arquitectura Transformer y LLM, en el contexto del procesamiento de lenguaje natural para e-commerce.

2. Implementar pipelines completos de clasificación de texto que incluyan preprocesamiento, tokenización, obtención de embeddings y clasificación, utilizando: clasificación directa por *few-shot prompting* con modelos GPT (OpenAI API y Groq), embeddings OpenAI (`text-embedding-3-small`) con clasificadores KNN y Random Forest, el modelo pre-entrenado BETO con fine-tuning, embeddings FastText con red neuronal densa, y la replicación de la solución ganadora con LSTM/GRU.
3. Comparar las métricas de rendimiento (accuracy, F1-score) de los modelos implementados y de la solución ganadora, para identificar los factores técnicos y de datos que explican sus diferencias en el problema de categorización del Meli Challenge 2019.

2 Marco Teórico

Este capítulo presenta los fundamentos conceptuales que sustentan los modelos evaluados en el trabajo. El campo del NLP ha atravesado tres generaciones de paradigmas: las **redes neuronales recurrentes** (RNN, LSTM, GRU), dominantes hasta 2017; la arquitectura **Transformer** con pre-entrenamiento y fine-tuning, que transformó el campo a partir de 2018; y los **grandes modelos de lenguaje** (LLM), que habilitan la clasificación directa mediante prompting. Comprender esta evolución es necesario para interpretar las diferencias de rendimiento observadas entre los modelos implementados.

2.1 Conceptos Básicos de Machine Learning para Clasificación

Esta sección describe algunos conceptos básicos del Machine Learning que son utilizados en este trabajo.

2.1.1 Aprendizaje Automático

Siguiendo el marco conceptual establecido por Mitchell (1997), se define el Machine Learning como: “A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E ”¹. Esta definición establece tres componentes fundamentales: la experiencia (E), que corresponde a los datos de entrenamiento; la tarea (T), que define qué tipo de problema se busca resolver; y la medida de rendimiento (P), que cuantifica qué tan bien el programa realiza la tarea.

2.1.2 Tarea de Clasificación

La clasificación es un tipo de tarea de aprendizaje supervisado en la cual el objetivo es asignar una etiqueta o categoría a una instancia de entrada basándose en sus características. En este tipo de tarea, el algoritmo aprende a partir de ejemplos etiquetados (experiencia E) para construir un modelo que pueda predecir la categoría correcta para nuevas instancias no vistas durante el entrenamiento. En el contexto del Meli Challenge 2019, la tarea consiste en clasificar productos de e-commerce en categorías específicas basándose en sus títulos y descripciones, donde cada producto representa una instancia y cada categoría representa una clase posible.

¹Mitchell, T. M. (1997). *Machine Learning*. McGraw Hill.

2.1.3 Métricas de Evaluación

Las métricas de evaluación cuantifican qué tan bien un modelo clasifica datos nuevos. En aprendizaje supervisado, estas métricas se calculan siempre sobre un **conjunto de test** —instancias etiquetadas que el modelo no vio durante el entrenamiento— para estimar el rendimiento real sobre datos no vistos [2].

Para la tarea de clasificación multiclase del Meli Challenge 2019, se emplean las siguientes métricas estándar [3]:

Accuracy (exactitud): proporción de predicciones correctas sobre el total de predicciones realizadas en el conjunto de test:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

Fuente: Jurafsky & Martin (2024), Cap. 4 [3].

donde TP (verdaderos positivos), TN (verdaderos negativos), FP (falsos positivos) y FN (falsos negativos) se computan por clase. La accuracy puede resultar engañosa en problemas con clases desbalanceadas, donde las categorías más frecuentes dominan el cómputo global.

Precision y Recall: para una clase c , la Precision indica qué proporción de las instancias predichas como c pertenecen efectivamente a esa clase; el Recall indica qué proporción de las instancias que realmente son c fueron identificadas correctamente por el modelo.

F1-Score: media armónica de Precision y Recall, especialmente útil cuando las clases están desbalanceadas:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (2)$$

En problemas multiclase, estas métricas se calculan por clase y luego se promedian —*macro-average* (igual peso a cada clase) o *weighted-average* (peso proporcional al número de instancias)— para obtener una métrica global comparable entre modelos.

2.1.4 Generalización

El desafío principal de un algoritmo de Machine Learning es performar bien en datos nuevos, es decir, no vistos durante el entrenamiento. Esta cualidad es conocida como *generalización*.

Los factores que determinan qué tan bien funcionará un algoritmo de Machine Learning pueden dividirse en dos categorías principales [2]: 1) hacer que el training error sea pequeño, y 2) hacer que la brecha entre training error y test error sea pequeña. Estos dos factores corresponden a los dos desafíos centrales en Machine Learning: *underfitting* y *overfitting*.

El *underfitting* ocurre cuando el modelo no es capaz de obtener un training error suficientemente bajo. Esto sucede cuando la capacidad del modelo es insuficiente para capturar la complejidad subyacente de los datos. En el contexto de clasificación de productos del Meli Challenge 2019, un modelo con *underfitting* podría no aprender los patrones necesarios para distinguir entre categorías similares, como por ejemplo, diferenciar entre “celulares” y “tablets”.

El *overfitting* ocurre cuando la brecha entre training error y test error es demasiado grande, es decir, cuando el modelo tiene un training error muy bajo pero un test error significativamente mayor. Esto sucede cuando el modelo tiene demasiada capacidad y memoriza los detalles específicos del conjunto

de entrenamiento en lugar de aprender patrones generalizables. En el problema de clasificación de categorías de productos, un modelo con overfitting podría memorizar títulos específicos del conjunto de entrenamiento y fallar al clasificar productos similares pero no idénticos en el conjunto de test.

La *capacidad* de un modelo se refiere a su habilidad para ajustarse a una amplia variedad de funciones. Un modelo con poca capacidad puede tener dificultades para ajustarse al conjunto de entrenamiento (underfitting), mientras que un modelo con demasiada capacidad puede memorizar el conjunto de entrenamiento sin generalizar bien (overfitting). La capacidad debe estar alineada con la complejidad del problema: para el Meli Challenge 2019, que involucra más de 1,500 categorías diferentes con patrones semánticos complejos en títulos y descripciones de productos, se requieren modelos con suficiente capacidad para capturar estas relaciones, pero sin excederla de manera que se produzca overfitting.

La relación entre la capacidad del modelo y los errores de entrenamiento y generalización se ilustra en la Figura 1, donde se observa cómo el error de entrenamiento disminuye con el aumento de la capacidad, mientras que el error de generalización alcanza un mínimo en un punto óptimo y luego aumenta debido al overfitting.

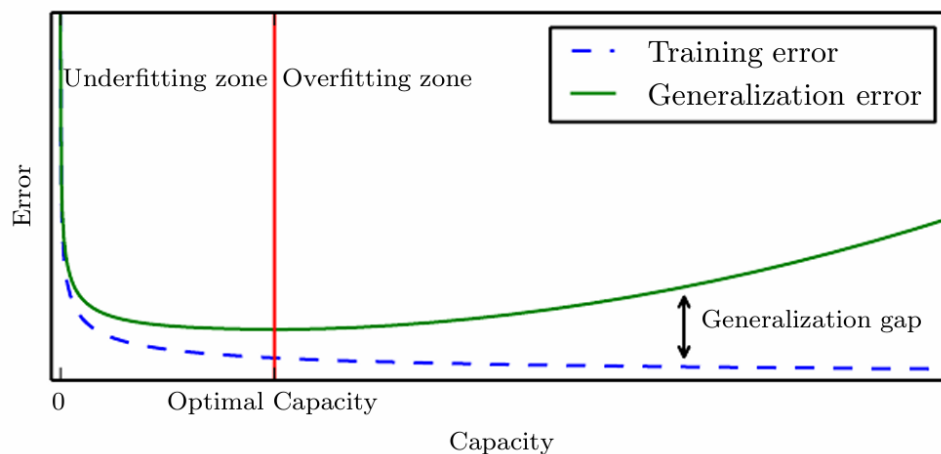


Figura 1: Relación entre la capacidad del modelo y los errores de entrenamiento y generalización. La zona de underfitting se caracteriza por altos errores en ambos conjuntos, mientras que la zona de overfitting muestra un bajo error de entrenamiento pero un alto error de generalización, con una brecha creciente entre ambos. Fuente: Goodfellow et al. (2016) [2].

2.2 Procesamiento de Lenguaje Natural (NLP)

El Procesamiento de Lenguaje Natural (NLP, por sus siglas en inglés *Natural Language Processing*) es una disciplina que se sitúa en la intersección de la lingüística computacional, la inteligencia artificial y el aprendizaje automático. Su objetivo es dotar a las computadoras de la capacidad de comprender, interpretar y generar lenguaje humano de manera útil [3]. Jurafsky y Martin (2024) definen el NLP como el conjunto de técnicas computacionales para analizar y representar textos de lenguaje natural a uno o más niveles de análisis lingüístico, con el propósito de lograr un procesamiento similar al humano para una variedad de tareas [3]. Esto abarca desde el nivel léxico (palabras) hasta el semántico (significado) y el pragmático (uso en contexto).

Las tareas de NLP pueden organizarse en un espectro de complejidad creciente: en el extremo más simple se encuentran la tokenización y el etiquetado morfológico (*POS tagging*); en un nivel intermedio,

tareas como la clasificación de texto, el análisis de sentimientos y la extracción de entidades nombradas; y en el nivel más complejo, la traducción automática, la generación de texto y la respuesta a preguntas. Este trabajo se focaliza en la **clasificación de texto**, tarea central del Meli Challenge 2019: dado el título de un producto en lenguaje natural, el objetivo es aprender una representación vectorial que capture su significado semántico y, a partir de ella, asignarlo a la categoría correcta entre más de 1.500 opciones del catálogo de Mercado Libre.

2.2.1 *Desafíos del Lenguaje Natural*

El lenguaje natural presenta desafíos que no aparecen en otros dominios del aprendizaje automático. Entre los más relevantes para este trabajo se destacan:

- **Ambigüedad léxica:** Una misma palabra puede tener múltiples significados según el contexto (p. ej., “banco” puede referirse a una institución financiera o a un asiento). En el catálogo de Mercado Libre, términos como “funda” o “tapa” pueden corresponder a categorías muy distintas.
- **Variabilidad léxica:** Un mismo concepto puede expresarse de múltiples formas (“celular”, “smartphone”, “teléfono móvil”). Los títulos de productos en e-commerce presentan alta variabilidad: abreviaturas, errores ortográficos, mezcla de idiomas y uso de códigos de modelo.
- **Dependencia del contexto:** El significado de una palabra depende de las palabras que la rodean. Los modelos que no capturan contexto (como TF-IDF o Word2Vec estático) pierden esta información.
- **Alta dimensionalidad del vocabulario:** Un corpus de e-commerce puede contener decenas de miles de términos únicos, incluyendo nombres de marcas, modelos y especificaciones técnicas.

Estos desafíos motivan la evolución desde representaciones esparsa (Sección 2.4) hacia modelos contextuales profundos basados en la arquitectura Transformer (Sección 2.5).

2.2.2 *Evolución de Paradigmas en la Clasificación de Texto*

La clasificación automática de texto ha sido abordada desde múltiples perspectivas en la literatura, con una evolución clara de paradigmas que estructura también el benchmark de este trabajo.

Hasta 2017, las **redes neuronales recurrentes** (RNN) constituían el estado del arte en modelado de secuencias, tal como documenta la introducción de Vaswani et al. (2017), que describe los modelos recurrentes como la referencia establecida en transducción de secuencias y modelado del lenguaje antes de proponer el Transformer [4]. Hochreiter y Schmidhuber (1997) introdujeron las redes LSTM para resolver el problema del gradiente que se desvanece en redes estándar [7], y Cho et al. (2014) extendieron este paradigma con las GRU [8]. La solución ganadora del Meli Challenge 2019 es representativa de este enfoque: combina embeddings FastText con capas LSTM y GRU, alcanzando una accuracy de 0.917 sobre el dataset completo.

A partir de 2018, la arquitectura **Transformer** y el paradigma de *pre-entrenamiento* y *fine-tuning* transformaron el campo. Como señalan Devlin et al. (2019) en la introducción de BERT, el pre-entrenamiento de modelos de lenguaje ha demostrado ser efectivo para mejorar numerosas tareas de NLP [5]: durante el pre-entrenamiento el modelo se entrena sobre datos no etiquetados mediante objetivos de modelado

de lenguaje; durante el fine-tuning, los parámetros pre-entrenados se ajustan con datos etiquetados de la tarea específica. Este paradigma es aplicable a distintas tareas simplemente conectando las entradas y salidas correspondientes, sin rediseñar la arquitectura. BETO [6] aplica este paradigma al español y es el modelo central del Modelo 2 de este trabajo.

Más recientemente, los **grandes modelos de lenguaje** (LLM) han consolidado un tercer paradigma: modelos pre-entrenados a escala masiva capaces de resolver tareas mediante instrucciones en lenguaje natural (*prompting*), sin fine-tuning, aprovechando capacidades emergentes que surgen con la escala [12]. Este trabajo evalúa dos variantes de este paradigma: clasificación directa por *few-shot prompting* con modelos GPT (Modelo 1) y uso de embeddings de alta calidad producidos por la API de OpenAI como representaciones para clasificadores externos (Modelo 3).

Esta taxonomía de tres generaciones —redes recurrentes, Transformers con fine-tuning, y LLM— articula el diseño comparativo del presente trabajo y orienta la lectura del Marco Teórico que sigue.

2.3 Deep Learning y Redes Neuronales Profundas

2.3.1 Redes Neuronales Profundas (Deep Feedforward Networks)

Las redes neuronales *feedforward* —también llamadas redes multicapa (*multilayer perceptrons*, MLP)— son el modelo central del Deep Learning [2]. Su objetivo es aproximar una función f^* : para un clasificador, $y = f^*(\mathbf{x})$ mapea una entrada $\mathbf{x}$ a una categoría y ; la red aprende el mapeo $y = f(\mathbf{x}; \theta)$ ajustando los parámetros θ que producen la mejor aproximación a f^* . La arquitectura consiste en capas de neuronas sucesivas: una capa de entrada, una o más capas ocultas y una capa de salida. Cada conexión entre neuronas tiene un *peso* (*weight*) asociado y cada neurona tiene un *sesgo* (*bias*); pesos y sesgos son los parámetros aprendibles del modelo θ [2]. En cada neurona se aplica una *función de activación* no lineal a la suma ponderada de sus entradas, lo que permite a la red aprender relaciones complejas entre variables; en la capa de salida, para clasificación multiclase, se utiliza la función *softmax*, que transforma los valores de salida en una distribución de probabilidad sobre las clases posibles. El algoritmo de aprendizaje determina cómo utilizar las capas ocultas para aproximar mejor f^* ; dado que los datos de entrenamiento no especifican una salida deseada para cada capa oculta en particular, éstas se denominan *capas ocultas* [2]. En estas redes la información fluye en una sola dirección durante la inferencia; cuando se incorporan conexiones de retroalimentación se obtienen las redes neuronales recurrentes (RNN), descritas a continuación.

El entrenamiento se realiza mediante iteraciones de dos pasos [2]. En el paso de *forward propagation*, la entrada $\mathbf{x}$ se procesa capa por capa y la predicción resultante se compara con la etiqueta real para calcular un error escalar. En el paso de *backpropagation*, ese error se distribuye hacia atrás a través de la red, proveyendo a cada neurona una medida de su contribución al error total; con esta información, el algoritmo de optimización ajusta los pesos y sesgos para reducir el error y mejorar la precisión del modelo en las siguientes predicciones. Este proceso se repite iterativamente —alternando pases hacia adelante y hacia atrás— hasta que el modelo converge. El algoritmo que aplica los ajustes de parámetros en cada iteración es el descenso por gradiente estocástico (SGD, Sección 2.3.2).

El término *Deep Learning* designa las redes con múltiples capas ocultas, capaces de aprender representaciones jerárquicas de los datos [2]: cada capa transforma la representación anterior en una abstracción de mayor complejidad, construyendo conceptos elaborados a partir de conceptos más simples. La ventaja central de estas arquitecturas radica en su capacidad de aprender representaciones directamen-

te a partir de datos crudos, sin necesidad de *feature engineering* manual [2]. En el contexto del Meli Challenge 2019, esto implica que el modelo descubre por sí mismo qué características de los títulos de productos son discriminativas para la clasificación, sin que el investigador deba definirlas a priori.

2.3.2 Descenso por Gradiente Estocástico y Función de Pérdida

El descenso de gradiente (GD) es un algoritmo de optimización que minimiza iterativamente una función objetivo. En el contexto del Machine Learning, esta técnica se utiliza durante la fase de entrenamiento de un modelo supervisado para minimizar la función de pérdida o costo, que mide qué tan lejos están las predicciones del modelo de los valores reales de los datos [2]. Un ejemplo común de función de costo es el error cuadrático medio.

El algoritmo calcula el gradiente de la pérdida con respecto a los parámetros del modelo y da un paso en la dirección opuesta para reducirla. La tasa de aprendizaje α controla el tamaño de cada paso. Este proceso se repite iterativamente hasta alcanzar la convergencia. Si α es demasiado grande, el algoritmo puede oscilar y converger a un mínimo local en lugar del mínimo global; una tasa de aprendizaje bien calibrada es esencial para lograr la convergencia.

En la práctica, el término *Stochastic Gradient Descent* (SGD) se usa para referirse al *descenso de gradiente de minilotes* (*mini-batch gradient descent*): en lugar del conjunto de entrenamiento completo —costoso computacionalmente— o de una sola muestra —ruidoso—, se utiliza un pequeño subconjunto (*minibatch*) en cada actualización [29, 2]. El gradiente resultante es el promedio de las pérdidas de ese minilote, lo que reduce el ruido y conduce a una convergencia más estable. El SGD estricto sobre una sola muestra rara vez se usa en la práctica; las bibliotecas PyTorch y TensorFlow denominan “SGD” a sus optimizadores aunque internamente emplean minilotes [29]. La actualización de parámetros en cada paso está dada por:

$$\theta = \theta - \alpha \nabla_{\theta} J(\theta; x^{(i)}, y^{(i)}) \quad (3)$$

donde $(x^{(i)}, y^{(i)})$ representa un par (o minilote) del conjunto de entrenamiento y α es la tasa de aprendizaje.

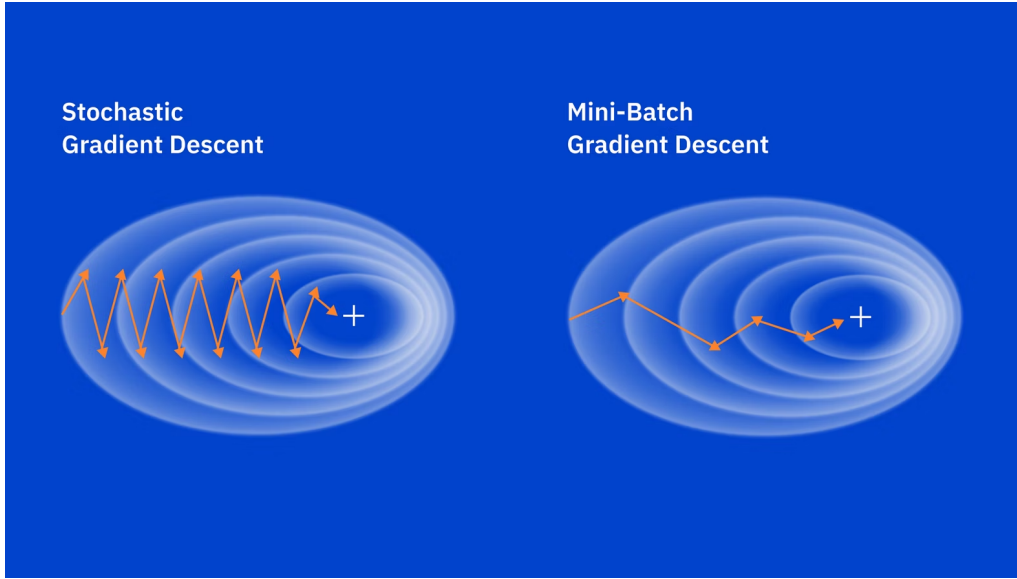


Figura 2: Cómo aumentar el tamaño de la muestra de los datos de entrenamiento reduce las oscilaciones y el ruido en la trayectoria de optimización. Fuente: IBM [29].

Para problemas de clasificación, la función de pérdida estándar es la **entropía cruzada** (*cross-entropy*). Desde la perspectiva estadística, el modelo define una distribución de probabilidad $p(y | \mathbf{x}; \theta)$ sobre las clases posibles; el principio de *máxima verosimilitud* establece que los parámetros óptimos son aquellos que maximizan la probabilidad asignada a las clases correctas en los datos de entrenamiento [2]. Maximizar esa verosimilitud es equivalente a minimizar la entropía cruzada entre la distribución predicha y la distribución real:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^K y_{ic} \log \hat{p}_{ic} \quad (4)$$

donde $y_{ic} \in \{0, 1\}$ indica si la instancia i pertenece a la clase c , y $\hat{p}_{ic}$ es la probabilidad predicha por el modelo. La entropía cruzada penaliza de forma asimétrica: cuando el modelo concentra su confianza en clases incorrectas (probabilidad baja para la clase correcta), la penalización $-\log \hat{p}_{ic^*}$ crece rápidamente; cuando el modelo asigna alta probabilidad a la clase correcta, la pérdida tiende a cero. En el Meli Challenge 2019, con más de 1.500 categorías, la entropía cruzada es la función de pérdida estándar para clasificación multiclase [2].

2.3.3 LSTM (Long Short-Term Memory)

Las redes neuronales recurrentes (RNN) fueron diseñadas para procesar secuencias, manteniendo un estado oculto que se actualiza en cada paso temporal. Sin embargo, las RNN estándar sufren el problema de *vanishing gradient*: los gradientes se vuelven exponencialmente pequeños al propagarse hacia atrás en secuencias largas, impidiendo que el modelo aprenda dependencias de largo alcance.

Hochreiter y Schmidhuber (1997) propusieron las redes **LSTM** (*Long Short-Term Memory*) para resolver este problema [7]. Una celda LSTM incorpora tres compuertas (*gates*) que regulan el flujo de información:

- **Compuerta de olvido (*forget gate*):** Decide qué información del estado anterior debe descartarse.

- **Compuerta de entrada (*input gate*):** Determina qué nueva información se almacena en el estado de celda.
- **Compuerta de salida (*output gate*):** Controla qué parte del estado de celda se expone como salida.

Este mecanismo permite a las LSTM capturar dependencias a larga distancia en el texto. En la solución ganadora del Meli Challenge 2019, se utilizaron capas CuDNNLSTM (una implementación acelerada por GPU de LSTM) para clasificar los títulos de productos.

2.3.4 GRU (*Gated Recurrent Unit*)

Cho et al. (2014) propusieron las **GRU** (*Gated Recurrent Unit*) como una variante simplificada de LSTM [8]. Las GRU combinan las compuertas de olvido y entrada en una única *compuerta de actualización*, y eliminan el estado de celda separado, manteniendo solo el estado oculto. Esto reduce el número de parámetros y el costo computacional respecto a LSTM, con rendimiento comparable en muchas tareas.

La solución ganadora del Meli Challenge 2019 empleó capas CuDNNGRU en combinación con CuDNNLSTM, aprovechando la complementariedad de ambas arquitecturas para capturar patrones secuenciales en los títulos de productos.

2.4 Representación del Texto: Embeddings

Para que los modelos de Machine Learning puedan procesar texto, es necesario convertirlo en representaciones numéricas. La evolución de estas representaciones es central para entender las diferencias de rendimiento entre los modelos evaluados en este trabajo.

2.4.1 De la Representación Esparsa a los Embeddings Densos

Las representaciones tradicionales como *bag of words* o TF-IDF producen vectores de alta dimensionalidad (del tamaño del vocabulario, potencialmente decenas de miles de dimensiones) y son *esparsas*: la mayoría de sus componentes son cero. Además, no capturan relaciones semánticas entre palabras: “celular” y “smartphone” producen vectores ortogonales aunque sean sinónimos.

Los *embeddings* (o incrustaciones vectoriales) son representaciones densas y de baja dimensionalidad (típicamente entre 50 y 1024 dimensiones) que se aprenden a partir de los datos. La idea clave es la hipótesis distribucional: palabras que aparecen en contextos similares tienden a tener significados similares [3]. De esta forma, los embeddings capturan relaciones semánticas y sintácticas que las representaciones esparsas no pueden.

2.4.2 Word2Vec y FastText

Word2Vec [10] fue uno de los primeros modelos en aprender embeddings de palabras de forma eficiente a partir de grandes corpus. Propone dos arquitecturas: *Continuous Bag of Words* (CBOW), que predice una palabra a partir de su contexto, y *Skip-gram*, que predice el contexto a partir de una palabra. Los vectores resultantes capturan analogías semánticas notables (p. ej., el vector de “rey” – “hombre” + “mujer” es cercano al vector de “reina”).

Sin embargo, Word2Vec tiene una limitación importante: produce un único vector por palabra, independientemente del contexto, y no puede representar palabras fuera del vocabulario de entrenamiento (OOV, *out-of-vocabulary*). En el dominio de e-commerce, esto es problemático: los títulos de productos contienen frecuentemente nombres de modelos, marcas y especificaciones técnicas poco frecuentes o nunca vistas durante el entrenamiento.

FastText [9] resuelve el problema de OOV representando cada palabra como la suma de vectores de sus *n*-gramas de caracteres. Por ejemplo, la palabra “celular” se representa mediante los *n*-gramas “cel”, “elu”, “lul”, “ula”, “lar” (entre otros). Así, palabras no vistas durante el entrenamiento pueden representarse si comparten subunidades con el vocabulario conocido. Esta propiedad es especialmente valiosa para el español, que tiene rica morfología, y para el dominio de productos de e-commerce con alta variabilidad léxica. En el Modelo 4 (Sección 4.4), se utiliza FastText en combinación con una red neuronal para la clasificación.

2.4.3 *Embeddings Contextuales*

Una limitación fundamental de Word2Vec y FastText es que producen representaciones *estáticas*: cada token tiene un único vector independientemente de su contexto oracional. La palabra “banco” tiene el mismo vector tanto en “fui al banco a cobrar” como en “me senté en el banco del parque”.

Los *embeddings contextuales* resuelven este problema produciendo representaciones que dependen del contexto completo de la oración. Modelos como ELMo [11] y, posteriormente, los basados en la arquitectura Transformer (BERT, GPT), generan vectores distintos para un mismo token según las palabras que lo rodean. En el problema de clasificación del Meli Challenge 2019, esto permite distinguir, por ejemplo, usos distintos de términos polisémicos en títulos de productos. Los embeddings contextuales se describen en detalle en la siguiente sección, junto con la arquitectura Transformer que los hace posibles.

2.5 Arquitectura Transformer

Los embeddings de texto utilizados en clasificación, búsqueda y bases vectoriales suelen producirse con *encoders* basados en la arquitectura Transformer. Esta subsección resume los conceptos y etapas típicas de ese pipeline siguiendo la arquitectura propuesta por Vaswani et al. (2017) [4], que introdujo el mecanismo de autoatención como componente central. Estos conceptos sustentan, en mayor o menor medida, modelos como BERT y los embeddings de APIs como la de OpenAI utilizados en la etapa de laboratorio.

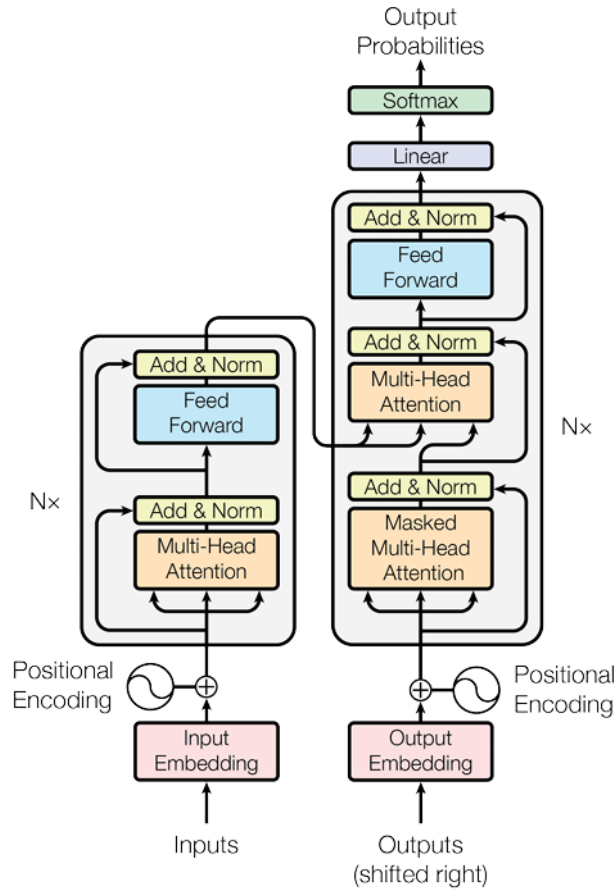


Figura 3: Arquitectura original del modelo Transformer [4]. El módulo *encoder* (izquierda) procesa la secuencia de entrada y produce representaciones contextuales; el módulo *decoder* (derecha) genera la secuencia de salida. Los modelos de clasificación como BERT utilizan únicamente el encoder; los modelos generativos como GPT utilizan únicamente el decoder.

2.5.1 Tokenización

La tokenización es el proceso de dividir el texto en unidades más pequeñas o *tokens*. Los tokens pueden ser palabras, subpalabras o caracteres. En modelos clásicos como Word2Vec se usaba vocabulario a nivel de palabra, lo que limita el manejo de palabras fuera de vocabulario (OOV) y hace crecer el modelo con el tamaño del vocabulario.

Entre las técnicas de *subpalabras* utilizadas en este proyecto cabe destacar dos familias. Por un lado, **FastText** (Modelo 4) define las subpalabras como *n-gramas de caracteres* de cada palabra. El vector de la palabra se obtiene como la suma de los vectores de sus *n-gramas*; así se pueden representar palabras OOV que comparten *n-gramas* con el corpus de entrenamiento. Por otro lado, los modelos basados en Transformer utilizan *tokenización por subpalabras*: un enfoque intermedio que mantiene las palabras frecuentes como un único token y descompone las palabras raras en fragmentos con significado, equilibrando el tamaño del vocabulario con la cobertura del corpus [28].

El algoritmo **Byte-Pair Encoding (BPE)** parte de una fase de *pre-tokenización* que divide el corpus de entrenamiento en palabras y cuenta la frecuencia de cada una. Con ese conteo se construye un *vocabulario base* formado por todos los símbolos únicos del corpus (típicamente caracteres individuales). A continuación, BPE aprende iterativamente *reglas de fusión*: en cada paso identifica el par de símbo-

los adyacentes que aparece con mayor frecuencia y los combina en un nuevo símbolo, que se agrega al vocabulario. Este proceso se repite hasta que el vocabulario alcanza el tamaño deseado, que es un hiperparámetro definido antes del entrenamiento. El tamaño final del vocabulario es igual al tamaño del vocabulario base más el número de combinaciones aprendidas [28]. La variante **Byte-level BPE**, adoptada por GPT-2 y sus sucesores, utiliza los 256 bytes posibles como vocabulario base, lo que garantiza que cualquier texto sea representable sin producir tokens desconocidos; GPT-2 alcanza un vocabulario de 50.257 tokens (256 bytes + 1 token especial + 50.000 combinaciones), y modelos modernos multilingües superan los 100.000.

WordPiece, empleado por BERT y BETO, sigue una lógica similar a BPE pero con un criterio de fusión distinto: en lugar de elegir el par de símbolos más frecuente, elige en cada paso el par cuya fusión *maximiza la probabilidad* de los datos de entrenamiento [28]. Intuitivamente, WordPiece evalúa si la ganancia de unir dos símbolos justifica perder la flexibilidad de tenerlos separados. El resultado son vocabularios de aproximadamente 30.000 tokens. La elección del esquema de tokenización afecta el tamaño del modelo, el manejo de OOV y la calidad final de los embeddings.

2.5.2 *Embedding Lookup (Búsqueda de Embeddings)*

Una vez tokenizado el texto, cada token se mapea a un vector denso inicial mediante una tabla de embeddings (*embedding lookup*). Estos vectores no son aleatorios: se aprenden en la fase de pre-entrenamiento del modelo y constituyen el punto de partida para que las capas posteriores capturen el significado de cada token en contexto.

2.5.3 *Codificación Posicional (Positional Encoding)*

La arquitectura Transformer no tiene noción intrínseca del orden de los tokens. Para incorporar la posición en la secuencia, se suman *codificaciones posicionales* a los embeddings iniciales [4]. Así el modelo puede distinguir, por ejemplo, “casa blanca” de “blanca casa”. Esta etapa permite procesar la secuencia de forma no secuencial (a diferencia de RNN/LSTM), lo que facilita el paralelismo y el entrenamiento a escala.

2.5.4 *Mecanismo de Autoatención (Self-Attention)*

El mecanismo de autoatención es la contribución central de Vaswani et al. (2017) [4]. Permite que cada token “consulte” al resto de los tokens de la secuencia y pondere su influencia según la relevancia. Así se obtienen representaciones que dependen del contexto completo (p. ej., la misma palabra tiene distinto vector según la oración).

En la práctica, para cada posición se calcula una combinación ponderada de las representaciones de todas las posiciones; los pesos se obtienen a partir de productos escalares entre vectores de consulta (*query*) y clave (*key*), normalizados con softmax. La atención puede representarse formalmente como:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (5)$$

donde Q , K , V son las matrices de consultas, claves y valores, y d_k es la dimensión de las claves. En la práctica, el Transformer utiliza *multi-head attention*: múltiples cabezales de atención en paralelo, cada uno enfocado en diferentes aspectos de las relaciones entre tokens.

2.5.5 Agregación y Pooling

Dentro de una capa Transformer, la salida del mecanismo de atención se agrega (p. ej. se suma con conexión residual y se normaliza) y se obtiene un vector por token que ya incorpora contexto. Para tareas que requieren *un único vector por documento o por título* (como clasificación o búsqueda por similitud), se aplica una etapa adicional de *pooling* sobre los vectores de los tokens: por ejemplo, promediar los vectores de todos los tokens (*mean pooling*), o usar el vector asociado a un token especial como [CLS] en BERT. Ese vector único es el embedding que se indexa o se usa como entrada a un clasificador.

2.5.6 Red Neuronal Feed-Forward

Tras las operaciones de atención (y opcionalmente más capas apiladas), los vectores por token se pasan por una red neuronal feed-forward (capas lineales con no linealidad), compartida para todas las posiciones. Esta red refina las representaciones y contribuye a la capacidad del modelo de capturar patrones más abstractos. La salida final de la última capa (antes o después del pooling, según el diseño) es el embedding que se utiliza para comparación o clasificación.

En resumen, el flujo típico es: *tokenización* → *embedding lookup* → + *codificación posicional* → (repetido) *autoatención* + *agregación/residual* + *feed-forward* → *pooling* (si se requiere un vector por texto). Los modelos de embedding como `text-embedding-3-small` de OpenAI no divulgan su arquitectura interna, pero se apoyan en principios análogos para producir vectores densos a partir de texto.

2.6 Modelos Pre-entrenados para NLP

Un avance clave en NLP fue el paradigma de *pre-entrenamiento* y *fine-tuning*: entrenar un modelo de gran capacidad sobre vastos corpus de texto no etiquetado (pre-entrenamiento), y luego adaptar ese modelo a una tarea específica con datos etiquetados (fine-tuning). Este paradigma permite que modelos entrenados con grandes recursos computacionales sean reutilizados para tareas con pocos datos etiquetados.

2.6.1 BERT

BERT (*Bidirectional Encoder Representations from Transformers*) fue propuesto por Devlin et al. (2019) [5] y representa un hito en NLP. A diferencia de los modelos de lenguaje unidireccionales anteriores, BERT es un *encoder* bidireccional: procesa cada token teniendo en cuenta tanto el contexto izquierdo como el derecho simultáneamente. Esto se logra mediante el pre-entrenamiento con dos tareas: *Masked Language Model* (MLM), donde se enmascaran tokens aleatorios y el modelo debe predecirlos; y *Next Sentence Prediction* (NSP), donde el modelo debe determinar si dos oraciones son consecutivas en el texto original.

BERT produce embeddings contextuales de alta calidad. Para tareas de clasificación, el vector del token especial [CLS] (añadido al inicio de cada secuencia) se usa como representación del texto completo y se pasa a una capa clasificadora adicional. BERT-base tiene 110 millones de parámetros y requiere grandes cantidades de datos y cómputo para alcanzar su potencial, como se evidencia en el experimento del Modelo 2 (Sección 4.2).

2.6.2 BETO: BERT para el Español

BETO es una versión de BERT pre-entrenada exclusivamente sobre un corpus en español, propuesta por Cañete et al. (2020) [6]. Se entrenó con un corpus de aproximadamente 3GB de texto en español proveniente de Wikipedia, OpenSubtitles y otros recursos, utilizando la misma arquitectura que BERT-base. Al estar pre-entrenado en español, BETO captura las particularidades morfológicas, sintácticas y semánticas del idioma de forma nativa, lo que lo hace especialmente adecuado para el Meli Challenge 2019, cuyos títulos de productos están mayoritariamente en español.

En el Modelo 2 (Sección 4.2), BETO se utiliza con fine-tuning sobre el dataset del Meli Challenge. Los experimentos muestran que el rendimiento de BETO es fuertemente dependiente del tamaño del conjunto de entrenamiento: con 70,000 registros la accuracy es de 0.47, mientras que con el dataset completo (10M registros) asciende a 0.88.

2.6.3 GPT y Embeddings de OpenAI

GPT (*Generative Pre-trained Transformer*) es la familia de modelos de OpenAI basados en la arquitectura Transformer *decoder*. A diferencia de BERT (encoder bidireccional), GPT procesa texto de izquierda a derecha y está optimizado para generación de texto. Los modelos de la familia GPT se pre-entrenan mediante modelado de lenguaje causal: predecir el siguiente token dado el contexto previo.

Para tareas de clasificación, OpenAI ofrece modelos de embedding dedicados (como `text-embedding-3-small`) que, aunque no divulgan su arquitectura interna, producen representaciones vectoriales de texto de alta calidad utilizables como features para clasificadores externos. En el Modelo 3 (Sección 4.3) se utilizan embeddings de OpenAI (`text-embedding-3-small`) como representación de los títulos de productos; el Modelo 1 (Sección 4.1), en cambio, emplea modelos GPT directamente como clasificadores mediante prompting.

2.7 Grandes Modelos de Lenguaje (LLM)

2.7.1 Definición y Escala

Los *Grandes Modelos de Lenguaje* (*Large Language Models*, LLM) son modelos de lenguaje basados en la arquitectura Transformer entrenados con volúmenes masivos de datos y parámetros en el orden de los miles de millones [12]. La escala es un factor diferencial: Zhao et al. (2023) documentan que a partir de cierto umbral de parámetros emergen capacidades que no están presentes en modelos más pequeños (fenómeno conocido como *emergent abilities*), incluyendo razonamiento en pocos ejemplos (*few-shot learning*) y seguimiento de instrucciones.

Modelos representativos incluyen GPT-3 (175B parámetros), GPT-4, LLaMA, y PaLM, entre muchos otros. En términos de uso de memoria y cómputo, entrenar un LLM desde cero requiere recursos fuera del alcance de la mayoría de las instituciones académicas; sin embargo, su uso vía API o mediante fine-tuning de modelos pre-entrenados es cada vez más accesible.

2.7.2 Pre-entrenamiento y Fine-tuning

El ciclo de vida de un LLM contempla dos etapas principales. En el *pre-entrenamiento*, el modelo se entrena sobre un corpus masivo de texto (libros, páginas web, código, artículos científicos) mediante

objetivos de modelado de lenguaje. En esta etapa se adquiere el conocimiento lingüístico y factual general. En el *fine-tuning*, el modelo pre-entrenado se ajusta sobre un conjunto de datos de la tarea específica (en este caso, clasificación de categorías de productos), lo que permite adaptar los pesos del modelo al dominio de aplicación [5]. Una variante moderna del fine-tuning es el *Instruction Tuning* y el aprendizaje por refuerzo con retroalimentación humana (RLHF), que alinean el comportamiento del modelo con las instrucciones del usuario.

2.7.3 Encoder vs. Decoder: Relevancia para Clasificación

Una distinción arquitectónica fundamental entre los LLM es si actúan como *encoders* o *decoders*. Los modelos encoder (como BERT/BETO) procesan la secuencia completa de forma bidireccional y son especialmente adecuados para tareas de comprensión, como clasificación y extracción de información. Los modelos decoder (como GPT) generan texto de forma autoregresiva y son adecuados para tareas de generación.

En el contexto de este trabajo, esta distinción atraviesa los tres paradigmas evaluados: BETO (encoder) se ajusta mediante fine-tuning supervisado sobre el dataset del Meli Challenge para clasificar directamente (Modelo 2); en el Modelo 3, la API de OpenAI (`text-embedding-3-small`) genera representaciones vectoriales densas de los títulos de productos, que luego se usan como features para clasificadores externos como KNN y Random Forest; y en el Modelo 1, los modelos GPT actúan directamente como clasificadores mediante *few-shot prompting*, sin necesidad de embeddings ni entrenamiento adicional.

2.8 Métricas de Evaluación

Las métricas utilizadas en este trabajo —accuracy, precision, recall y F1-score— se definen en la Sección 2.1.3 del Marco Teórico. En el contexto del Meli Challenge 2019, se reporta la **accuracy** como métrica principal de comparación con la solución ganadora del benchmark, complementada con el **F1-score macro** para evaluar el rendimiento equilibrado entre las más de 1.500 categorías del catálogo.

2.9 Trabajos Relacionados

2.9.1 Clasificación de texto en e-commerce

La propuesta de este trabajo contrasta con la solución ganadora del Meli Challenge 2019, que combina LSTM/GRU con embeddings FastText, en dos dimensiones principales. En primer lugar, incorpora paradigmas que no existían o no estaban disponibles en 2019: modelos Transformer con fine-tuning (BETO) y embeddings de LLM vía API (OpenAI `text-embedding-3-small`), lo que permite evaluar si los avances del campo desde entonces se traducen en mejoras concretas sobre este problema específico. Devlin et al. (2019) demostraron que el paradigma de pre-entrenamiento y fine-tuning supera a los enfoques basados en redes recurrentes en la mayoría de las tareas de comprensión del lenguaje [5], hipótesis que este trabajo pone a prueba en el dominio concreto de categorización de productos en español. En segundo lugar, complementa el enfoque recurrente original incorporando FastText combinado con una red neuronal densa —en lugar de LSTM/GRU— como punto de comparación dentro del mismo paradigma de embeddings de subpalabras, permitiendo aislar el impacto de la arquitectura recurrente respecto de la representación del texto.

2.9.2 Categorización automática de productos en e-commerce

La categorización automática de productos en e-commerce presenta un desafío de clasificación multiclase con características propias del dominio: la ausencia de recursos léxicos preexistentes que capturen marcas, modelos y abreviaturas técnicas obliga a construir representaciones directamente desde los datos, sin poder apoyarse en ontologías o diccionarios de propósito general [22]. Este trabajo parte de ese diagnóstico e incorpora representaciones basadas en embeddings densos y modelos Transformer, que capturan relaciones semánticas que las representaciones léxicas tradicionales no pueden expresar, evaluando si esos avances resuelven las limitaciones identificadas en el mismo tipo de problema.

2.9.3 Clasificación multiclase de texto a gran escala

Dos líneas de trabajo son especialmente relevantes como antecedentes. Por un lado, Cañete et al. (2020) desarrollaron BETO, el primer modelo BERT pre-entrenado exclusivamente en español, demostrando que el fine-tuning sobre un modelo monolingüe obtiene mejores resultados que los modelos BERT entrenados en corpus multilingües en la mayoría de las tareas de NLP en español [6]. Este trabajo utiliza BETO como modelo central del benchmark, adoptando directamente esa metodología de pre-entrenamiento monolingüe y fine-tuning supervisado. Por otro lado, Nam et al. (2014) demostraron que redes neuronales simples equipadas con técnicas modernas de entrenamiento —ReLU, dropout y AdaGrad— con cross entropy como función de pérdida alcanzan o superan el estado del arte en clasificación multiclase de texto a gran escala [25]. Este trabajo extiende esa línea incorporando paradigmas posteriores —embeddings contextuales y Transformers— evaluando su comportamiento en el dominio específico de categorización de productos de e-commerce en español.

2.9.4 Benchmarks comparativos de modelos de NLP

Los benchmarks comparativos de modelos de NLP han sido un área de investigación activa desde la introducción de GLUE (Wang et al., 2018) [27], que evaluó sistemáticamente BERT y sus variantes en tareas diversas de comprensión del lenguaje estableciendo un protocolo de comparación sobre el mismo conjunto de datos y métricas. Zhao et al. (2023) [12], en su survey de LLM, proporcionan una síntesis actualizada del estado del arte en clasificación de texto, incluyendo el impacto de la escala del modelo y del volumen de datos de pre-entrenamiento. La metodología de este trabajo sigue ese enfoque de benchmarking sistemático, aplicándolo a un problema específico —categorización de productos de e-commerce en español— con modelos de distintas generaciones y paradigmas.

3 Metodología e Implementación

3.1 Entorno de Experimentación

Los experimentos se ejecutaron en dos entornos computacionales según los requerimientos de cada modelo. Los modelos que demandan mayor capacidad de cómputo —como BETO con fine-tuning sobre el dataset completo— se ejecutaron en un entorno local con GPU NVIDIA RTX 3060 (12GB VRAM). Los modelos basados en embeddings de API (OpenAI text-embedding-3-small y prompting con GPT) se obtuvieron mediante llamadas a servicios en la nube de OpenAI y Groq. Para el experimento con GPT-OSS-120B se utilizó el nivel gratuito de **GroqCloud** [18], una plataforma que ejecuta LLM

sobre *Language Processing Units* (LPU), una nueva categoría de procesador diseñada específicamente para inferencia de modelos de lenguaje que opera hasta 10 veces más eficientemente en términos energéticos que las GPU convencionales, a la vez que ofrece mayor velocidad de inferencia [20]. Adicionalmente, se utilizó Google Colaboratory (Colab) como entorno de experimentación complementario en la nube pública de Google, que provee acceso a recursos de cómputo GPU/TPU sin infraestructura propia, especialmente útil para prototipado y validación de pipelines. El repositorio del código es público: https://github.com/mari-fra/tp_final.

3.2 Dataset

Se utiliza el dataset público del Meli Challenge 2019 [13], proporcionado por Mercado Libre para la competencia de Kaggle. El dataset completo en portugués y español contiene 20 millones de registros. El subconjunto en español, sobre el que se centra este trabajo, comprende **10 millones de registros** con dos columnas principales: título del producto (texto libre en español) y categoría (etiqueta de clase). El dataset presenta más de **1.588 categorías con distribución desbalanceada**: las categorías más frecuentes concentran la mayor parte de los registros, mientras que muchas categorías tienen pocas decenas de ejemplos. Este desbalance es uno de los principales desafíos en aprendizaje automático sobre este tipo de datos, ya que los modelos tienden a favorecer las clases mayoritarias, degradando el rendimiento sobre las categorías menos frecuentes [23].

Para los experimentos con la API de OpenAI (costo por token), se utilizaron muestras estratificadas de 1.000 y 10.000 registros, aplicando muestreo estratificado por categoría para asegurar que las clases más relevantes estén representadas proporcionalmente. Para los modelos entrenados localmente o en Colab (BETO, FastText), se realizaron ensayos con muestras más amplias llegando hasta el dataset completo.

3.3 Preprocesamiento

Las variables de entrada son los títulos de productos (texto libre). La variable de salida es la categoría del producto (clasificación multiclase con más de 1.500 clases). El preprocesamiento incluye normalización de caracteres y manejo de valores numéricos —normalizados pero no eliminados, siguiendo la estrategia de la solución ganadora que demostró que preservar los números mejora el rendimiento en categorías donde las especificaciones técnicas son discriminativas (p. ej., “disco duro 1TB” vs. “disco duro 500GB”). Se aplicó además un filtrado de categorías con menos de 20 ejemplos para garantizar la estratificación del split train/validación/test.

Una decisión central en el pipeline es la **estrategia de tokenización**, que varía según el modelo: los modelos basados en FastText (Modelo 4 y solución ganadora) utilizan n-gramas de caracteres, lo que permite representar términos no vistos mediante subunidades; los modelos Transformer (BETO, embeddings OpenAI) utilizan tokenización por subpalabras mediante esquemas BPE/WordPiece, con vocabularios que varían según la arquitectura: BERT utiliza WordPiece con vocabularios de ~30.000 tokens (entrenado sobre inglés), mientras que GPT-3 usa Byte-level BPE con ~50.000 tokens; los modelos GPT más modernos y multilingües a menudo superan los 100.000 tokens de vocabulario para cubrir múltiples idiomas y scripts.

Para representar un título completo como un único vector a partir de los embeddings de sus palabras constituyentes, se aplica *mean pooling*: el vector del título p se obtiene como el promedio de los vectores

de cada uno de sus M tokens, siguiendo la metodología de Kozareva (2015) [22]:

$$p = [e_1, \dots, e_d] \quad \text{donde} \quad e_i = \frac{1}{M} \sum_{j=1}^M e_i w_j \quad (6)$$

donde d es la dimensión del vector de embedding, e_i es el valor en la posición i del vector del título, y $e_i w_j$ es el valor en la posición i del vector de la palabra w_j . Este enfoque es utilizado en los modelos basados en embeddings estáticos (FastText). Los modelos Transformer (BETO) utilizan el vector del token especial [CLS] como representación del título completo, mientras que la API de OpenAI devuelve directamente el vector agregado.

3.4 Descripción de los Modelos Evaluados

3.4.1 Criterios de Selección y Marco Común

Los modelos evaluados fueron seleccionados para representar los tres enfoques descritos en el Marco Teórico: enfoques de clasificación directa por prompting con LLM (Modelo 1), modelos Transformer con fine-tuning (Modelo 2), embeddings contextuales de API con clasificadores externos (Modelo 3), y aprendizaje profundo con embeddings de subpalabras (Modelo 4), en comparación con la solución ganadora basada en LSTM/GRU (2019).

Para todos los modelos que incluyen una capa de clasificación neuronal, la función de salida es la función **softmax**, que transforma los logits de salida en una distribución de probabilidad sobre las K categorías posibles [2]:

$$\hat{y}_k = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}, \quad k = 1, \dots, K \quad (7)$$

donde z_k es el logit de la clase k y K es el número total de categorías (1.588 en el dataset del Meli Challenge 2019). La clase predicha es aquella con mayor probabilidad: $\hat{y} = \arg \max_k \hat{y}_k$.

El entrenamiento de los modelos neuronales minimiza la función de pérdida de **entropía cruzada** (*cross-entropy*), que mide la discrepancia entre la distribución predicha y la distribución real de clases [2, 25]:

$$L = - \sum_{k=1}^K y_k \cdot \log(\hat{y}_k) \quad (8)$$

donde y_k es 1 si la instancia pertenece a la clase k y 0 en caso contrario (*one-hot encoding*), y $\hat{y}_k$ es la probabilidad predicha para la clase k por la función softmax. La minimización de esta función mediante Descenso de Gradiente Estocástico (SGD) ajusta iterativamente los parámetros del modelo para mejorar la predicción sobre el conjunto de entrenamiento. Esta función de pérdida es utilizada por BETO (Modelo 2), FastText con red neuronal (Modelo 4) y la solución ganadora con LSTM/GRU.

3.4.2 Resumen Comparativo de Enfoques

Cuadro 1: Resumen comparativo de los modelos evaluados

Modelo	Representación	Clasificador	Notebook
Modelo 1	Prompting con GPT (OpenAI/Groq)	Generación directa	01
Modelo 2	BETO (fine-tuning)	Capa softmax	02, 02_all
Modelo 3	text-embedding-3-small	KNN, Random Forest	03, 03b
Modelo 4	FastText + Red Neuronal	Red neuronal densa	04
Solución 2019	FastText + LSTM/GRU	Softmax	(ganador)

3.5 Modelo 1 — Clasificación por Prompting con GPT (OpenAI)

El Modelo 1 emplea modelos GPT de OpenAI como clasificadores directos mediante *few-shot prompting*: el modelo recibe el título del producto junto a una lista de categorías y genera directamente el nombre de la categoría predicha, sin entrenamiento adicional sobre el dataset del Meli Challenge 2019. Se evaluaron tres variantes de modelos GPT con el mismo protocolo experimental, lo que permite comparar rendimiento y costo.

3.5.1 Descripción del Enfoque

Los LLM han demostrado capacidad para realizar aprendizaje en contexto (*in-context learning*) mediante *few-shot prompting*, técnica que permite resolver tareas condicionando la respuesta del modelo a un prompt construido con unos pocos ejemplos de entrada-salida, sin necesidad de fine-tuning y aprovechando el conocimiento adquirido durante el pre-entrenamiento [12]. En este trabajo se aplica para clasificar títulos de productos en español y portugués en categorías en inglés, combinando comprensión multilingüe y conocimiento del dominio de e-commerce.

El pipeline de clasificación consta de los siguientes pasos:

1. Para cada título de producto, se construye un prompt que incluye: (a) un *system message* que define el rol del modelo como clasificador; (b) tres ejemplos etiquetados (*few-shot examples*) de las categorías más frecuentes del dataset; (c) la lista completa de categorías válidas en la muestra; y (d) el título del producto a clasificar.
2. El modelo responde con el nombre de la categoría predicha, sin texto adicional.
3. La predicción se compara con la etiqueta real para calcular las métricas de evaluación.

El *system message* utilizado en los tres experimentos fue:

“You are a product classifier for MercadoLibre product titles. Titles may be in Spanish or Portuguese. Always classify them into one of the provided categories, which are in English. Return only the category name, without explanations or extra text. If no category seems perfect, choose the closest match from the list. Do not invent new categories.”

Los tres ejemplos *few-shot* corresponden a títulos reales de las tres categorías más frecuentes del dataset:

1. ‘Butaca 6 Cuotas Sin Interes!! Para Auto Bebesit Hasta 25kg’ → BABY_CAR_SEATS
2. ‘Cafetera De Filtro Hamilton Beach 12 Tazas’ → COFFEE_MAKERS
3. ‘Calça Feminina Pra Mulher Gravidas Dois Bolso Cintura Alta’ → PANTS

3.5.2 Preparación del Dataset

Se tomó una muestra aleatoria reproducible de 1.000 registros del dataset completo (`random_state=42`) y se aplicó un filtro mínimo de 3 ejemplos por categoría, resultando en 247 registros distribuidos en 73 categorías. Este tamaño reducido de muestra se justifica por el costo por token de las APIs utilizadas: cada solicitud de clasificación consume aproximadamente 569 tokens (instrucciones del sistema + ejemplos *few-shot* + lista de 73 categorías + título del producto).

3.5.3 Plataformas de Inferencia: OpenAI API y Groq

Los experimentos con GPT-5-nano y GPT-5-mini se ejecutaron directamente sobre la **OpenAI API**, el servicio de inferencia en la nube de OpenAI que da acceso a sus modelos propietarios con facturación por token consumido.

El experimento con GPT-OSS-120B se ejecutó sobre **Groq** [18], una plataforma en la nube que permite ejecutar distintos LLM (LLaMA, GPT, entre otros) con mayor velocidad y menor latencia que las implementaciones convencionales basadas en GPU, gracias al uso de unidades de procesamiento de lenguaje propias (*Language Processing Units*, LPU).

3.5.4 Modelos Evaluados

- **GPT-5-nano** (OpenAI API): modelo más compacto de la familia GPT-5, optimizado para bajo costo y alta velocidad de inferencia.
- **GPT-5-mini** (OpenAI API): modelo de capacidad intermedia de la familia GPT-5, con mayor rendimiento que *nano*.
- **GPT-OSS-120B** (Groq): modelo de código abierto de 120 mil millones de parámetros publicado por OpenAI, ejecutado sobre Groq bajo el identificador `openai/gpt-oss-120b` [19] en el nivel gratuito de la plataforma.

3.5.5 Resultados

Cuadro 2: Resultados comparativos — Modelo 1 (Clasificación por Prompting con GPT)

Modelo	Plataforma	Registros evaluados	Accuracy	F1-Score (macro)
GPT-5-nano	OpenAI API	247	0.866	0.844
GPT-OSS-120B	Groq API	201*	0.910	0.894
GPT-5-mini	OpenAI API	247	0.919	0.915

*El experimento con GPT-OSS-120B se interrumpió al alcanzar el límite diario gratuito de la plataforma Groq (199.566 de 200.000 tokens por día consumidos). Los 201 registros procesados representan el 81 % de la muestra.

Los tres modelos alcanzan una accuracy superior a 0.86 sobre la muestra de 73 categorías, siendo GPT-5-mini el de mejor rendimiento (accuracy 0.919, F1-macro 0.915). Cabe señalar que estos resultados se obtuvieron sobre tan solo 73 de las 1.588 categorías del dataset original, lo que reduce sustancialmente la dificultad del problema respecto a la clasificación completa. Esta limitación se aborda en el análisis comparativo de la Sección 5.

3.5.6 Análisis de Viabilidad Económica

El costo por inferencia es una limitación central de este enfoque a escala industrial. Con las 1.588 categorías del dataset completo, cada solicitud requeriría aproximadamente 10.000 tokens entre instrucciones, contexto y categorías posibles. La Tabla 3 muestra el costo estimado para clasificar los 20 millones de registros del dataset completo, basado en los precios publicados por OpenAI [21].

Cuadro 3: Estimación de costo para clasificar el dataset completo (20M registros, ~10.000 tokens/solicitud)

Modelo	Precio	Costo estimado
GPT-5-nano	\$0,0001 / 1k tokens	~\$20.000
GPT-4o-mini	\$0,15 / 1M tokens	~\$30.000
GPT-4o	\$2,50 / 1M tokens	~\$500.000
GPT-4-turbo	\$10,00 / 1M tokens	~\$2.000.000

Aun con el modelo más económico, clasificar el dataset completo tendría un costo estimado de \$20.000, lo que evidencia que el enfoque de *prompting* con LLM no es viable en producción para este problema sin ajustes significativos en el diseño de la solución.

3.6 Modelo 2 — BETO

3.6.1 Configuración del Entrenamiento

El entrenamiento se realizó en un equipo con GPU NVIDIA RTX 3060 (12GB VRAM), lo cual requirió configuraciones específicas para optimizar el uso de memoria y acelerar el proceso:

- **Mixed Precision (FP16):** Reducción del uso de VRAM en aproximadamente 40 % y aceleración del entrenamiento en ~20 %
- **Batch size:** 16 por dispositivo con gradient accumulation de 2 pasos (batch efectivo de 32)
- **Warmup ratio:** 10 % de los steps totales para estabilizar el entrenamiento inicial
- **Early stopping:** Paciencia de 2 epochs para evitar overfitting

3.6.2 Dependencia de Accelerate

Un aspecto técnico relevante es la dependencia de la librería `accelerate` de Hugging Face. El Trainer de Hugging Face Transformers delega todas las operaciones de bajo nivel a esta librería, incluyendo:

- Movimiento de tensores entre CPU y GPU
- Implementación de mixed precision (FP16/BF16)
- Manejo de gradient accumulation
- Soporte para entrenamiento distribuido multi-GPU

Esta separación arquitectónica se realizó para mantener la librería `transformers` más liviana, permitir configuraciones avanzadas sin modificar el código base, y reutilizar la lógica de aceleración en otras librerías del ecosistema Hugging Face (como `diffusers`).

3.6.3 Preparación del Dataset

Para el split estratificado del dataset se requirió un filtro mínimo de 20 ejemplos por categoría. Esto se debe a que el proceso de división (90 % train, 5 % validación, 5 % test) con estratificación necesita suficientes ejemplos en cada categoría para garantizar representación en todos los conjuntos. Categorías con menos de 20 ejemplos pueden resultar en splits donde alguna categoría quede con solo 1 ejemplo, causando errores en la estratificación.

3.6.4 Requisitos de Software

La ejecución en GPU moderna (RTX 30xx/40xx) con CUDA 12.x requiere:

- PyTorch ≥ 2.6 (debido a vulnerabilidad de seguridad CVE-2025-32434 en versiones anteriores)
- `accelerate` $\geq 0.26.0$
- Drivers NVIDIA actualizados compatibles con CUDA 12.x

3.6.5 Validación de Extrapolabilidad de Resultados

Con el propósito de validar si los resultados obtenidos con una muestra reducida de 70,000 registros (notebook `02_tp_final_beto.ipynb`) eran extrapolables al dataset completo en español, se entrenó el mismo modelo BETO con el dataset completo de aproximadamente 10 millones de registros (notebook `02_tp_final_beto_all.ipynb`) utilizando la misma arquitectura y configuración de hiperparámetros.

Los resultados obtenidos muestran una diferencia en el rendimiento entre ambas configuraciones:

- **Muestra de 70,000 registros:** Accuracy de 0.46 en el conjunto de test
- **Dataset completo (~10M registros):** Accuracy de 0.88 en el conjunto de test

La diferencia observada de 42 puntos porcentuales indica que los resultados no son extrapolables de la muestra pequeña al dataset completo. Las posibles razones de esta discrepancia incluyen:

1. **Representatividad de la muestra:** Una muestra de 70,000 registros puede no capturar adecuadamente la distribución completa de categorías y patrones presentes en el dataset completo, considerando que el dataset original contiene más de 1,500 categorías diferentes.
2. **Balance de clases:** Con una muestra pequeña, algunas categorías pueden estar subrepresentadas o ausentes, afectando la capacidad del modelo para aprender patrones generalizables.
3. **Overfitting en muestra pequeña:** Con menos datos, el modelo puede memorizar patrones específicos de la muestra en lugar de aprender representaciones generalizables, resultando en un rendimiento reducido cuando se evalúa en el conjunto completo.
4. **Capacidad del modelo:** BETO es un modelo de 110 millones de parámetros que requiere grandes cantidades de datos para alcanzar su potencial. Con 70,000 ejemplos, el modelo no tiene suficientes datos para aprender efectivamente las relaciones complejas entre texto y categorías.

Esta validación demuestra la importancia de evaluar modelos en datasets de tamaño similar al de producción, especialmente cuando se trabaja con modelos transformer de gran capacidad como BETO.

3.7 Modelo 3 — OpenAI Embeddings (text-embedding-3-small)

Se utilizó el modelo `text-embedding-3-small` de OpenAI para el problema de clasificación de categorías de productos. Esta sección describe la metodología del embedding, la solución implementada, los dos ensayos realizados y el uso de la Batch API.

3.7.1 Metodología del Embedding

Según la documentación de OpenAI [15], los embeddings son representaciones numéricas de texto que permiten medir la afinidad semántica entre dos fragmentos. El modelo `text-embedding-3-small` está pensado para búsqueda, clustering, recomendaciones, detección de anomalías y tareas de clasificación. La API recibe el texto (el título del producto) y devuelve un único vector denso de dimensión fija (1536 para `text-embedding-3-small`).

3.7.2 Descripción de la Solución

La solución consiste en: (1) obtener un vector de embedding por título mediante la API de OpenAI; (2) usar esos vectores como features para entrenar clasificadores supervisados. Se probaron Random Forest y K-Nearest Neighbors (KNN), con variantes de KNN: sin escalado, con `StandardScaler` y con reducción de dimensionalidad (PCA a 50 componentes). La evaluación se realiza sobre un conjunto de test estratificado; las métricas reportadas son accuracy (y, donde aplica, `classification report`).

3.7.3 Ensayo 1 (03_tp_final_embedding.ipynb)

- **Muestra:** 1.000 registros tomados al azar del dataset; filtro de al menos 3 ejemplos por categoría, resultando en 247 filas útiles.
- **API:** Llamadas síncronas a la API de embeddings (no Batch).
- **Train/valid:** 70 % entrenamiento, 30 % validación (172 y 75 muestras respectivamente).

- **Mejor resultado:** KNN con escalado (StandardScaler), accuracy 0.53 en validación.

3.7.4 Ensayo 2 (03_tp_final_embedding_2.ipynb)

- **Muestra:** 10.000 registros con muestreo estratificado por categoría; filtro técnico de al menos 20 ejemplos por categoría y luego filtro de al menos 2 ejemplos por clase para permitir split estratificado (70 % train / 30 % test).
- **API:** Batch API de OpenAI [16]: mismo modelo y misma lógica de embedding, con procesamiento asíncrono y descuento del 50 % en coste.
- **Train/test:** 70 % entrenamiento, 30 % test (~6.880 y ~2.949 muestras).
- **Mejor resultado:** KNN sin escalado, accuracy 0.48 en test; Random Forest 0.19; KNN con PCA 0.38.

La accuracy no mejoró en el Ensayo 2 a pesar de usar una muestra mayor (10k frente a ~247) porque el problema es más difícil: en el Ensayo 2 se conservan muchas más categorías, de modo que el número de clases es mucho mayor. A mayor número de clases, la tarea de clasificación es más difícil y la accuracy típicamente baja para el mismo modelo y la misma métrica.

3.7.5 Batch API

La Batch API de OpenAI [16] permite enviar conjuntos de peticiones de forma asíncrona. Para el endpoint `/v1/embeddings`, los parámetros de cada petición (modelo, texto de entrada) son los mismos que en la API síncrona; la diferencia está en el modo de envío y el coste. Ventajas relevantes para este proyecto: (1) descuento del 50 % respecto a la API estándar; (2) límites de tasa separados y más altos; (3) tiempo de completitud dentro de 24 horas. Para embeddings, el límite es de 50,000 entradas por lote; en el Ensayo 2 se usaron 10,000 títulos en un solo lote.

3.8 Modelo 4 — FastText con Red Neuronal

3.9 Solución Original del Meli Challenge 2019

3.9.1 Limitaciones de Reproducibilidad

La solución ganadora del 2019 [14] presenta problemas de reproducibilidad debido a:

1. Dependencias obsoletas:

- Python 3.6 / 3.7 (EOL - End of Life)
- TensorFlow 1.14
- CUDA 10.0 + cuDNN 7.6

2. Incompatibilidades técnicas:

- TensorFlow 1.14 no es compatible con GPUs modernas (RTX 30xx)
- CuDNNLSTM y CuDNNGRU requieren versiones específicas de CUDA/cuDNN que ya no están disponibles para drivers modernos

- Error: “No OpKernel was registered to support Op ‘CudnnRNN’”
- Estas capas no tienen fallback a CPU, por lo que el entrenamiento falla inmediatamente sin GPU funcional

3. Problemas con bibliotecas:

- Incompatibilidad gensim + smart_open
- SyntaxError con Python 3.6 por uso de `from __future__ import annotations`
- El paquete smart_open utiliza anotaciones de tipo de Python 3.10+, no soportadas en Python 3.6

4. Recursos computacionales:

- Embeddings de gran tamaño (~6 GB)
- FastText requiere mucha RAM durante el entrenamiento
- Entrenamiento extremadamente lento en CPU
- Costo computacional elevado incluso con las tecnologías modernas

3.9.2 Workarounds Implementados

Para poder evaluar los modelos, se implementaron las siguientes soluciones alternativas:

- Reemplazo de CuDNNLSTM/CuDNNGRU por LSTM/GRU estándar (permitiendo ejecución en CPU pero perdiendo la aceleración GPU específica de la solución original)
- Migración a TensorFlow 2.x o alternativas modernas
- Adaptación del código para ejecución en CPU
- Uso de versiones modernas de Python (3.8+) y bibliotecas compatibles

Nota importante: Estos workarounds permiten la ejecución funcional pero alteran la arquitectura original, haciendo imposible una replicación exacta de la solución ganadora del 2019 en entornos modernos.

3.10 Limitaciones del Estudio

El presente trabajo opera bajo tres limitaciones principales que deben considerarse al interpretar los resultados:

1. **Reproducibilidad parcial de la solución ganadora.** La replicación del modelo ganador del Meili Challenge 2019 no es exacta debido a incompatibilidades de dependencias (TensorFlow 1.14, CUDA 10.0) con hardware moderno. El reemplazo de CuDNNLSTM/CuDNNGRU por sus equivalentes estándar altera la arquitectura original y puede afectar el rendimiento respecto al reportado originalmente [26].

2. **Tamaño de muestra en experimentos con API.** Los experimentos con la API de OpenAI —tanto embeddings (Modelo 3) como prompting (Modelo 1)— están limitados en tamaño de muestra por razones de costo por token. Las métricas obtenidas sobre muestras de 247 a 10.000 registros no son directamente comparables con los resultados obtenidos sobre el dataset completo de 10 millones de registros.
3. **Especificidad del dominio.** Los resultados son específicos del dominio de e-commerce en español y portugués, con títulos de productos como texto de entrada y categorías en inglés como *output*. La generalización de las conclusiones a otros dominios o idiomas requiere validación adicional [22].

3.11 Resultados

3.11.1 Métricas de Evaluación

Cuadro 4: Comparación de Métricas por Modelo

Modelo	Descripción	Accuracy	Precision	Recall	F1-Score
Solución Original (2019)	Dataset completo, modelo ganador del Meli Challenge 2019	0.917	—	—	—
Modelo 1 (GPT-5-mini)	Muestra 247 registros, 73 categorías, few-shot prompting	0.919	—	—	0.915
Modelo 2 (BETO)	Muestra de 70,000 registros en español, BETO (BERT pre-entrenado)	0.47	—	—	—
Modelo 2 (BETO completo)	Dataset completo en español (~10M registros), BETO	0.88	—	—	0.84
Modelo 3 (Embeddings v1)	Muestra 1k → 247 filas, text-embedding-3-small, API síncrona, KNN escalado	0.53	—	—	—
Modelo 3 (Embeddings v2)	Muestra 10k estratificada, text-embedding-3-small, Batch API, KNN (mejor)	0.48	—	—	—
Modelo 4 (FastText NN)	Dataset completo en español (~10M registros), FastText con redes neuronales	0.81	—	—	—

Nota: Los valores “—” indican métricas no reportadas en el notebook correspondiente (precision/recall no calculados en todos los experimentos).

4 Discusión

5 Conclusiones y Trabajo Futuro

5.1 Conclusiones

5.2 Trabajo Futuro

6 Referencias

Referencias

- [1] Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
- [2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. <https://www.deeplearningbook.org>
- [3] Jurafsky, D., & Martin, J. H. (2024). *Speech and language processing* (3rd ed., online draft). <https://web.stanford.edu/~jurafsky/slp3/>
- [4] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008. <https://arxiv.org/abs/1706.03762>
- [5] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186. <https://arxiv.org/abs/1810.04805>
- [6] Cañete, J., Chaperon, G., Fuentes, R., Ho, J.-H., Kang, H., & Pérez, J. (2020). Spanish pre-trained BERT model and evaluation data. *Practical ML for Developing Countries Workshop, ICLR 2020*. <https://arxiv.org/abs/2308.02976>
- [7] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [8] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *Proceedings of EMNLP 2014*, 1724–1734. <https://arxiv.org/abs/1406.1078>
- [9] Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5, 135–146. <https://aclanthology.org/Q17-1010/>
- [10] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*. <https://arxiv.org/abs/1301.3781>
- [11] Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., & Zettlemoyer, L. (2018). Deep contextualized word representations. *Proceedings of NAACL-HLT 2018*, 2227–2237. <https://arxiv.org/abs/1802.05365>

- [12] Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., ... Wen, J.-R. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*. <https://arxiv.org/abs/2303.18223>
- [13] Mercado Libre. (2019). *MeLi data challenge 2019* [Kaggle competition]. <https://www.kaggle.com/c/mercadolibre-data-challenge-2019>
- [14] García, E. (2019). *MeLi data challenge 2019 — winner solution* [repositorio GitHub]. <https://github.com/eduagarcia/meli-challenge-2019>
- [15] OpenAI. (2024). *Text embeddings* [documentación de API]. <https://platform.openai.com/docs/guides/embeddings>
- [16] OpenAI. (2024). *Batch API* [documentación de API]. <https://platform.openai.com/docs/guides/batch>
- [17] Huang, J., & Chang, K. C.-C. (2023). Towards reasoning in large language models: A survey. *arXiv preprint arXiv:2303.13217*. <https://arxiv.org/abs/2303.13217>
- [18] Groq. (2024). *GroqCloud documentation*. <https://console.groq.com/docs/overview>
- [19] Groq. (2024). *GPT-OSS-120B model on GroqCloud* [documentación de modelo]. <https://console.groq.com/docs/model/openai/gpt-oss-120b>
- [20] Groq. (2024). *The Groq LPU explained* [blog oficial]. <https://groq.com/blog/the-groq-lpu-explained>
- [21] OpenAI. (2024). *OpenAI API pricing* [página oficial]. <https://openai.com/api/pricing/>
- [22] Kozareva, Z. (2015). Everyone likes shopping! Multi-class product categorization for e-commerce. *Human Language Technologies: The 2015 Annual Conference of the North American Chapter of the ACL*, 1329–1333. <https://aclanthology.org/N15-1147>
- [23] He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284. <https://doi.org/10.1109/TKDE.2008.239>
- [24] Manning, C. D., & Schütze, H. (1999). *Foundations of statistical natural language processing*. MIT Press.
- [25] Nam, J., Kim, J., Loza Mencía, E., Gurevych, I., & Fürnkranz, J. (2014). Large-scale multi-label text classification — revisiting neural networks. *Proceedings of ECML PKDD 2014*, 437–452. https://doi.org/10.1007/978-3-662-44851-9_28
- [26] Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d’Alché-Buc, F., Fox, E., & Larochelle, H. (2021). Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(1), 7459–7478. <https://jmlr.org/papers/v22/20-1364.html>

- [27] Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2018). GLUE: A multi-task benchmark and analysis platform for natural language understanding. *Proceedings of EMNLP 2018 Workshop BlackboxNLP*. <https://arxiv.org/abs/1804.07461>
- [28] HuggingFace. (2024). *Descripción general de los tokenizadores* [documentación de Transformers]. https://huggingface.co/docs/transformers/es/tokenizer_summary
- [29] IBM. (2024). *Stochastic gradient descent*. IBM Think. <https://www.ibm.com/mx-es/think/topics/stochastic-gradient-descent>

A Código y Repositorio

El código completo, notebooks y scripts están disponibles en: https://github.com/mari-fran/tp_final